

# Your agent doesn't need an LLM to plan



Twenty steps. **Twenty model calls, every single run** — nothing lets the agent notice it already solved this on attempt one.

MEET GATS

# Graph-Augmented Tree Search

## DEFAULT

### ✗ Model decides every step

Every step of execution asks the LLM what happens next — the most expensive, least consistent voice in the stack, deciding everything.

## GATS

### ✓ Model is the last resort

The agent builds a persistent world model and searches it. The LLM only steps in when nothing cheaper can answer.

THE SEARCH ITSELF COSTS NOTHING TO RUN · REINFORCEMENT-LEARNING LITERATURE SINCE THE 1990s

## THE CLAIM

# Separate deciding from producing.

"What should  
happen next"

"What does this  
action produce"

Separate the two, and the plan comes out dramatically cheaper and bit-exact identical every time.

## THE DEFAULT SHAPE

# Every step is a model call. Every run starts from nothing.

### COST

A twenty step task pays twenty times. A hundred step task pays a hundred.

### LATENCY

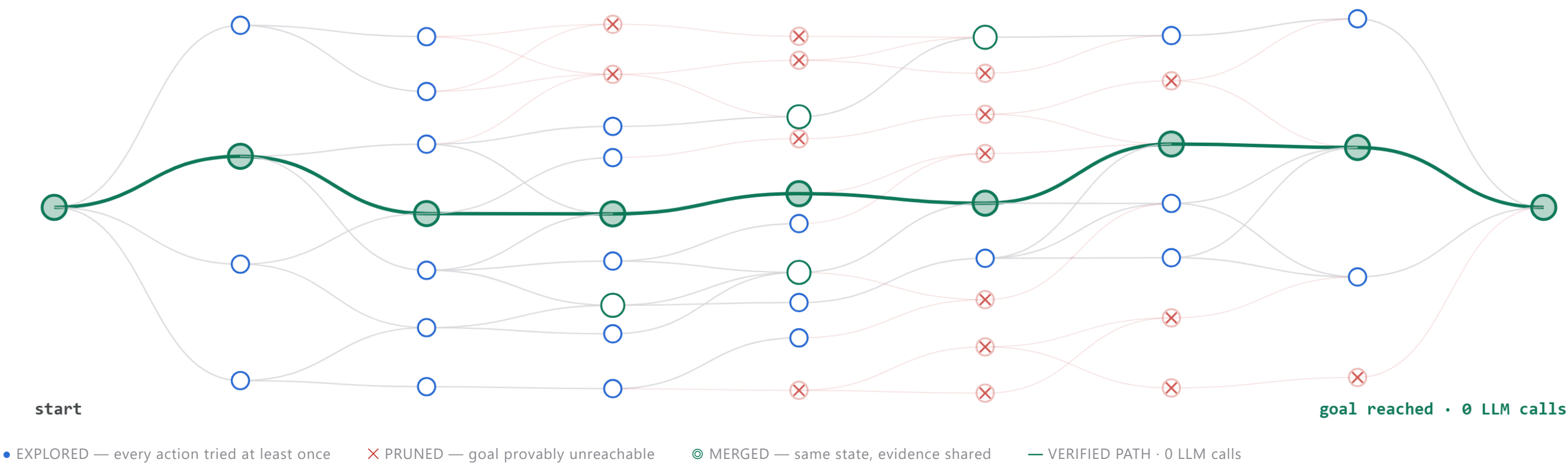
Planning latency is model latency, multiplied by depth.

### VARIANCE

Same task, same inputs, different plan. Nothing replays.

The model re-derives what it already worked out last run, because nothing  
was written down.

# Stop asking the model. Search the graph.



## THE ESCALATION LADDER

# Each rung fires only when the one below it cannot answer.

### L1 SYMBOLIC

**Free and exact.** Preconditions and effects already known. Covers most of what an agent does all day.

### L2 LEARNED

Per-state transition statistics mined from the agent's own execution history. Takes over once an action turns out to be stochastic.

### L3 GENERATIVE

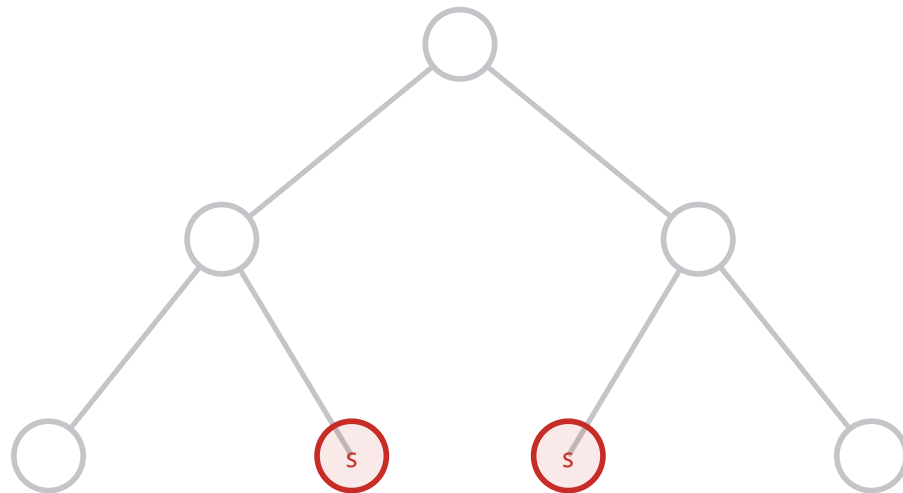
The LLM proposes effects for genuinely novel actions. The result is **cached**.

A novel situation costs at most one call, ever.

# Episode one pays. Episode two is cheaper.

The graph is written once and reused. On the frozen benchmark, reuse cut model queries **83%** with zero completion loss.

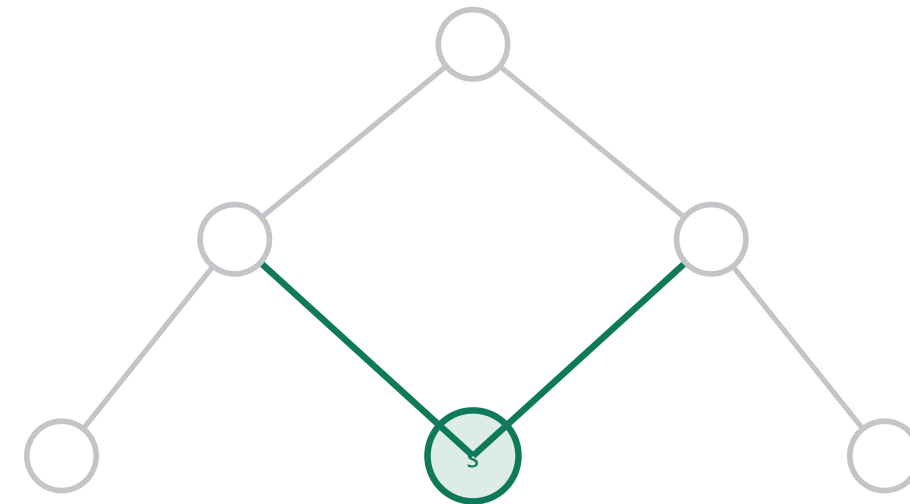
## NAIVE TREE SEARCH · NO MEMORY



same state, expanded twice

never collapses — cost multiplies below

## GATS · GRAPH MEMORY

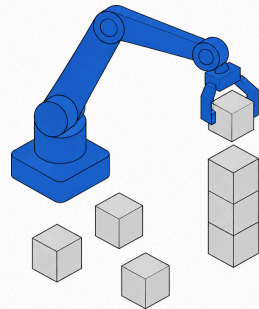


one node — evidence reused

canonical key = transposition table

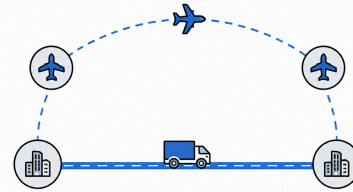
## THE BENCHMARK

# Three PlanBench domains, thirty percent masked.



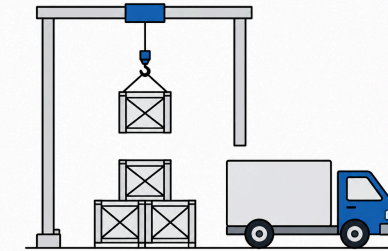
### BLOCKSWORLD

Stack labeled blocks. One-block arm.



### LOGISTICS

Trucks + planes, two-level network.



### DEPOTS

Hoists + trucks — the harder hybrid.

30% OF TRANSITION INFO WITHHELD · FORCES LAYERS L2 / L3 TO ENGAGE

# What holds up under audit.

**148/150**

GATS, masked-transition planning

**65/150**

matched bounded BFS control

**83%**

fewer model queries from graph reuse, no completion loss

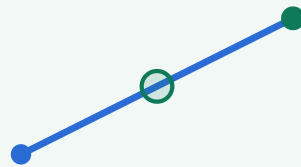
**2e-25**

McNemar p-value, GATS vs matched BFS

SO I TURNED PIECES OFF

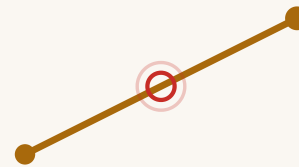
# Is the graph doing anything at all?

## FULL SYSTEM



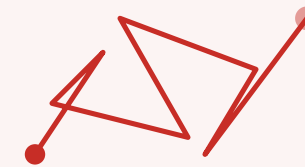
Heuristic points the way. Graph caches the repeats.

## NO GRAPH MEMORY



Same path. Every repeat re-queries the model.

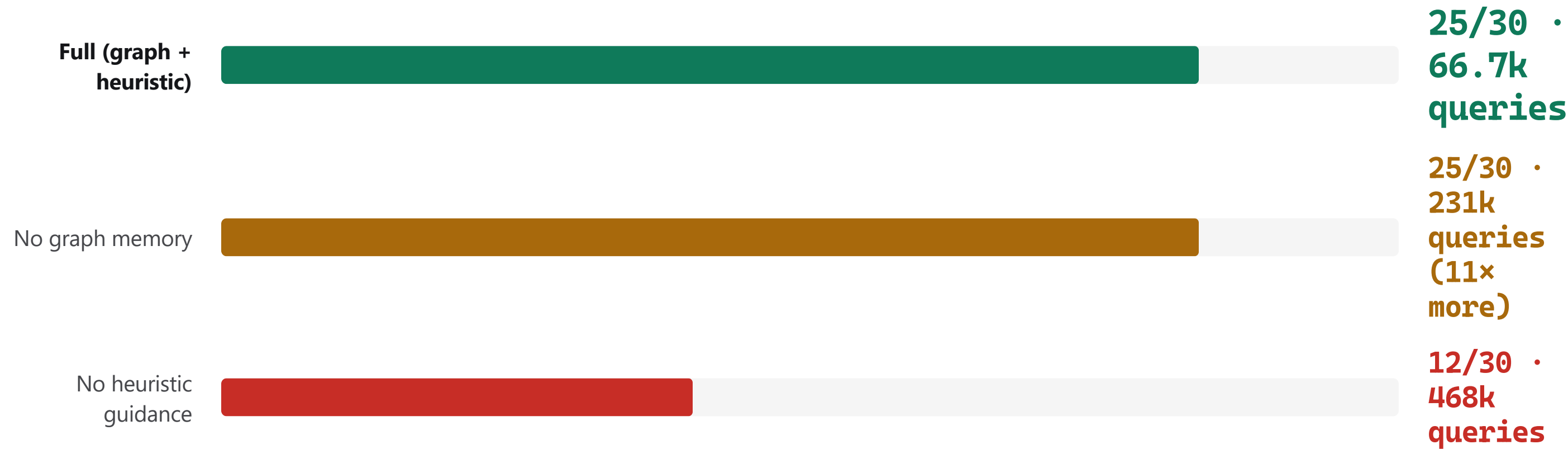
## NO HEURISTIC



No direction. Wanders until the budget runs out.

Same tasks, same budget, same predictor. Graph memory and heuristic guidance **disabled one at a time.**

# Turning pieces off, one at a time.



The heuristic decides if you get there. The graph decides what it costs.

WHY IT HOLDS

# Heuristic guidance drives success. Graph memory drives efficiency.

Disable the graph and success is unchanged, only the query count multiplies. Disable the heuristic and success collapses. **Two different jobs**, not one mechanism doing both.

WHERE THIS DOESN'T APPLY

# Three honest limits.

01

Needs a **specifiable action catalog**. Open-ended natural-language action spaces give you no graph to build.

02

The evidence is **narrow**: three PlanBench domains, one frozen split. Not evidence this generalizes everywhere.

03

The evaluation is **single-agent**. I am not claiming multi-agent results I do not have.

## TAKEAWAY

# The model call in your planning loop is a choice.

If your agent re-derives the same plan every run, it is paying for memory it could have written down.

[arXiv:2607.08894](https://arxiv.org/abs/2607.08894) · [github.com/MMWilliams/gats](https://github.com/MMWilliams/gats)

MAUREESE WILLIAMS · [linkedin.com/in/maureese-williams](https://linkedin.com/in/maureese-williams)